

Análisis técnico integral — Centro COP Escalable

Fecha: 31 de agosto de 2026 · **Estado:** migración incremental a Spring Boot iniciada.

1. Resumen ejecutivo

Centro COP es una plataforma clínica de odontología y psicología: gestión de pacientes, agenda, historia clínica, odontograma, psicología, reservas públicas, pagos, analítica, J48 e IA/3D. Es un monorepo con dos frontends Next.js, dos frontends Angular, una API NestJS, MongoDB/Redis y servicios Python. El objetivo vigente es centralizar todo el servidor en Spring Boot/Java y retirar los runtimes heredados sólo después de paridad funcional.

Hallazgo crítico: `web-public` y `web-dashboard` no son legados; son las aplicaciones Next.js principales y están declaradas en `render.yaml`. Frontend y PublicWeb son los Angular expresamente marcados como legacy. Eliminar Next.js hoy rompe la web pública y el panel desplegados.

2. Inventario y arquitectura actual

Componente	Tecnología	Función / estado
web-public	Next.js App Router	Landing, reserva, login, registro y cuenta; frontend principal.
web-dashboard	Next.js App Router	Panel clínico, pacientes, citas, psicología, J48 e IA; frontend principal.
Frontend / PublicWeb	Angular	Panel y portal alternativos; README raíz los clasifica legacy.
nest-migration	NestJS/Mongoose	API operacional: JWT, RBAC, catálogo, agenda, clínica, pagos, IA y compatibilidad.
services/* Python	FastAPI/ML	J48, recomendación, emoción, diagnóstico, DICOM e imagen 3D.
services/j48-service	Spring Boot/Weka	J48 existente, con pruebas y seguridad.
backend-spring	Spring Boot	Nuevo API de migración: IAM, pacientes, agenda, sedes y catálogo.

3. Backend, datos y seguridad

La persistencia operativa es MongoDB vía Mongoose; no hay evidencia de que el SQL PostgreSQL documentado sea el origen activo. Colecciones: organizaciones, sedes, usuarios, refresh tokens, profesionales, pacientes, citas, historia, odontogramas, psicología, snapshots/predicciones J48, IA médica, trabajos 3D, reseñas, catálogo y reservas/pagos.

Las relaciones son referencias por ID y los datos incluyen UUID tanto en forma string como BSON. La primera fase Spring conserva MongoDB y esos contratos. JWT Bearer, bcrypt, refresh rotation, roles y filtro por organización/sede son reglas obligatorias. CORS, Helmet, rate limiting, errores, Redis/BullMQ y Socket.IO requieren equivalencia antes del corte.

4. APIs y migración realizada

Contrato	Estado Spring	Reglas preservadas
/health, /health/live	Migrado	Salud del proceso.
/api/auth/login, /refresh	Migrado	bcrypt, JWT HS256, refresh token en Mongo.
/api/patients	Migrado	RBAC, organización/sede, paginación y búsqueda.
/api/appointments	Migrado	Profesionales, filtros, estados, claim y solapamiento.
/api/services y sedes	Migrado	Catálogo/ofertas/categorías y rutas públicas/administrativas.
Clínica, psicología, odontograma, público/pagos, IA, 3D, analítica	Pendiente	Siguen temporalmente en Nest/Python.

5. Riesgos y plan

- **Crítico: pagos, webhooks, datos clínicos y tokens no pueden migrarse sin pruebas de contrato y copia de Atlas.**
- **Crítico: los modelos Python no se pueden sustituir por stubs; deben portar inferencia a Java (Weka, ONNX Runtime Java/DJL según artefacto real) y compararse con datasets de regresión.**
- Alto: las agregaciones Mongo, los UUID BSON/string, WebSocket y trabajos asíncronos deben conservar semántica.
- Alto: valores de desarrollo y archivos temporales de login deben rotarse y excluirse antes de producción.

1. Completar módulos clínicos, público/pagos, analítica, IA y 3D en Spring.
2. Pruebas unitarias, integración Mongo/Redis y contratos Nest→Spring; ejecutar E2E de ambos frontends.
3. Despliegue paralelo/canary, respaldo y validación de datos.
4. Reconfigurar consumidores y retirar Nest/Python.
5. Eliminar sólo el frontend que se confirme reemplazado, actualizar Render/CI/Compose y verificar rollback.

6. Decisión de eliminación requerida

No se ha borrado ningún frontend. Para una eliminación segura hace falta decidir cuál queda: **A)** conservar Next.js y eliminar Angular legacy (Frontend, PublicWeb), coherente con la configuración actual; o **B)** eliminar Next.js y migrar/verificar toda su funcionalidad en Angular antes. La petición actual nombra Next.js, pero el repositorio lo identifica como activo, por lo que borrar sin esta decisión sería una pérdida funcional.